

CONTENTGEN – BLOG MANAGEMENT API

Project Documentation

A project documentation for an AI-assisted Blog Management Backend

Project title: AI BlogNest API

Team leader

LOGAPRIYA. T(222406687)

Team member

MAHALAKSHMI. D(222406688)

MAHALAKSHMI. N(222406689)

MEENALOSHINI. M(222406690)

MIDHUNA. P(222406691)

1. INTRODUCTION

ContentGen is a backend REST API project developed for creating and managing blog content. The application is built using Node.js and Express.js and uses MongoDB with Mongoose for storing and managing data. It also connects with Google Gemini AI for AI-assisted content generation and summarization.

The main purpose of this project is to make blog management easier by providing APIs for users, blogs, comments, categories and AI-based content features. The system also provides authentication and role-based access so that different users can perform different operations.

2. PROJECT OBJECTIVES

- To provide a simple backend API for blog creation and management.
- To store users, blogs and related information in MongoDB.
- To provide secure user authentication using JWT.
- To control access based on user roles such as Admin, Editor, Author and Reader.
- To provide AI-based blog generation and content summarization.
- To support blog comments, categories, tags and media information.
- To validate and sanitize user input for better security.

3. PROBLEM STATEMENT

Managing several blog posts manually can become difficult when there are many drafts, categories, comments and users. Content creators may also spend a lot of time preparing and summarizing

content. A centralized backend system is therefore needed to manage blog data, user permissions and AI-assisted content operations in an organized way.

4. PROPOSED SYSTEM

The proposed system, ContentGen, provides a centralized REST API for blog management. It separates the application into different layers such as routes, controllers, models and services. MongoDB is used as the database, while Mongoose handles schema definitions and database operations.

The system can be accessed through API tools such as Postman. Protected APIs check the user's authentication token and role before allowing restricted operations.

5. MAIN FEATURES

- User registration and login
- JWT-based authentication
- Role-based access control
- Create, read, update and delete blog operations
- Categories and tags
- Comment management and moderation
- AI blog generation
- Blog summarization
- Input validation and sanitization
- Centralized error handling
- Search and content management support

6. SYSTEM REQUIREMENTS

6.1 Software Requirements

- Operating System: Windows 10/11, macOS or Linux
- Node.js: Version 16 or above
- npm: Version 8 or above
- Express.js
- MongoDB
- Mongoose
- Postman or Thunder Client
- Visual Studio Code or another code editor

6.2 Hardware Requirements

- Processor: Intel Core i5 8th Generation or equivalent AMD processor
- RAM: Minimum 8 GB; 16 GB recommended
- Storage: At least 1 GB of available space

7. SYSTEM ARCHITECTURE

The application follows a layered backend architecture. A client such as Postman sends a request to the Express server. The request is checked by authentication and authorization middleware. The controller then processes the request and communicates with the model layer. Mongoose sends the required database operation to MongoDB. AI-related requests are handled through the AI service.

Basic flow:

Client → Express Server → Authentication / Middleware → Controller → Model → MongoDB

For AI operations:

Client → API Route → Controller → AI Service → Gemini AI → Response

8. DATABASE DESIGN

8.1 User Collection

- `_id` – Primary key (ObjectId)
- `name` – User name
- `email` – Required and unique email
- `password` – Hashed password
- `role` – User role, with reader as the default

8.2 Post Collection

- `_id` – Primary key (ObjectId)
- `userID` – Reference to the User collection
- `name` – Blog title
- `photo` – Image or media path
- `message` – Main blog content
- `likes` – Number of likes, default 0
- `timestamps` – Stores creation and update times

8.3 Relationships

- One User can create many Posts.
- One User can add many Comments.
- One Post can contain many Comments.

9. USER ROLES

- Admin – Manages users, roles, comments and system-level information.
- Editor – Reviews posts, manages publishing status and categories.
- Author – Creates and manages posts associated with the author's account.
- Reader – Reads published blogs and can add comments.

10. MVC ARCHITECTURE

10.1 Model

The Model layer defines the structure of data using Mongoose schemas. It specifies fields, validation rules and default values before data is stored in MongoDB.

10.2 Controller

The Controller receives requests from routes, checks the required information, performs the required operation through models and sends a JSON response.

10.3 View / API Layer

Since this is a backend API project, it does not use normal HTML views. The API routes act as the interface between the client and the controller.

11. PROJECT FOLDER SETUP

The project can be organized into a main project folder with a Server folder. Inside the Server folder, the basic backend structure contains the server file and separate folders for models, controllers and routes.

1. Create a project folder.
2. Create a Server folder inside it.
3. Open the Server folder in Visual Studio Code.
4. Open the terminal in VS Code.
5. Run the command: `npm init -y`
6. Create `server.js`.
7. Create models, controllers and routes folders.

12. BACKEND DEVELOPMENT

The backend server is created using Node.js and Express.js. The server file starts the application and connects the API routes. Controllers contain the main processing logic, while models define the MongoDB data structure.

The project also uses middleware for authentication, authorization, validation and other common backend operations.

13. MONGODB CONFIGURATION

8. Install Mongoose in the Node.js project.
9. Use the command: `npm install mongoose`
10. Create the MongoDB connection.
11. Create the required Mongoose schemas and models.
12. Make sure the database connection is established before performing database operations.

Mongoose is used as the ODM layer between the Node.js application and MongoDB. It helps define schemas, validate data and perform database operations.

14. API TESTING

Postman or the Thunder Client extension can be used to test the backend APIs. Start the backend server using the project's start command and then send requests to the required API endpoints.

- User Registration – tests creation of a new user.
- User Login – tests authentication and login.
- Create Blogs – tests creation of a new blog.
- Fetch Blogs – tests retrieving blog information.
- Generate AI Blogs – tests AI-assisted blog generation.
- Summarize Blogs – tests AI-assisted summarization.

15. SECURITY

- JWT is used for stateless authentication.
- Passwords are protected using bcrypt.js hashing.
- Role-based access prevents unauthorized operations.
- Input validation helps maintain correct data.
- Input sanitization helps reduce common injection and XSS risks.
- Rate limiting and structured logging are included as additional security measures.

16. ADVANTAGES

- Centralized blog management
- Easy API-based access
- Separate user roles and permissions
- MongoDB provides flexible document storage
- AI can reduce the time required for content creation
- Blog summarization can provide shorter versions of long content
- Layered architecture makes the backend easier to organize and maintain

17. CONCLUSION

ContentGen is an AI-assisted blog management backend that combines REST API development with MongoDB and Generative AI. The project provides basic blog management, user authentication, role-based access, comments and AI-based content features. The layered structure using routes, controllers, models and services makes the project easier to understand and maintain.

The project demonstrates how Node.js, Express.js, MongoDB, Mongoose, authentication and AI services can be combined to create a practical backend application.

18. PROJECT EXECUTION CHECKLIST

- Create the project and Server folders.
- Initialize the Node.js project using `npm init -y`.
- Install required packages.
- Configure MongoDB connection.
- Create models and schemas.
- Create controllers and routes.
- Configure authentication and authorization.
- Start the backend server.
- Test APIs using Postman or Thunder Client.
- Test registration, login, blog creation, blog fetching and AI features.

19. REFERENCE / PROJECT RESOURCES

The documentation is prepared from the supplied office project documentation and keeps the same project concepts while presenting them in a simpler student-style structure.

CONTENTS

1. Introduction
2. Project Objectives
3. Problem Statement
4. Proposed System
5. Main Features
6. System Requirements
7. System Architecture
8. Database Design
9. User Roles
10. MVC Architecture
11. Project Folder Setup
12. Backend Development
13. MongoDB Configuration
14. API Testing
15. Security
16. Advantages
17. Conclusion
18. Project Execution Checklist
19. Detailed Working of the System
20. API Request and Response Flow
21. AI Features
22. Testing and Expected Results
23. Limitations and Future Scope
24. Project Summary

19. DETAILED WORKING OF THE SYSTEM

ContentGen works as a backend application in which a client communicates with the server through REST API endpoints. The server receives a request, checks the required information and performs the requested operation.

19.1 User Registration

A new user can register by providing details such as name, email and password. The information is validated before storage. Passwords are protected using hashing.

19.2 User Login

During login, the supplied credentials are checked. After successful authentication, a JWT token is used for authenticated requests.

19.3 Creating a Blog

An authorized author can create a blog by sending the required information through the API. The controller processes the request and stores the blog in MongoDB.

19.4 Fetching Blogs

The retrieval API reads blog documents from MongoDB and returns them as JSON data. Published content can be consumed through the available routes.

19.5 Comment Management

Users can submit comments on posts. Authorized users can manage comment status such as approved, pending or spam.

19.6 Role Checking

Middleware checks authentication and role information before protected operations are allowed.

20. API REQUEST AND RESPONSE FLOW

The project follows a REST API request cycle, which separates the client from backend business logic.

13. Client sends an HTTP request.
14. Express receives the request.
15. Middleware checks authentication when required.
16. The request reaches the controller.
17. The controller validates and processes the request.
18. The model communicates with MongoDB when required.
19. The controller prepares the result.
20. The server returns a JSON response.

20.1 HTTP Methods

- GET – retrieving information.
- POST – creating information.
- PUT – updating information.
- DELETE – removing information where permission is available.

20.2 JSON Communication

The backend communicates using JSON-formatted data. This makes the API convenient to test using Postman or Thunder Client.

21. AI FEATURES

ContentGen integrates Google Gemini AI to reduce manual effort in preparing and summarizing blog content.

21.1 AI Blog Generation

The AI generation feature accepts a topic or suitable input and sends it to the AI service. The returned content can be used as a starting point for a blog.

21.2 Blog Summarization

The summarization feature processes longer content and produces a shorter summary.

21.3 AI Flow

User Request → API Route → Controller → AI Service → Gemini AI → Generated Result → API Response

21.4 Benefits

- Reduces time needed for an initial draft.
- Provides a structured starting point for content.
- Creates shorter versions of long content.
- Combines blog management with generative AI.

22. TESTING AND EXPECTED RESULTS

Testing is performed to check whether the backend endpoints work correctly. Postman or Thunder Client can be used to send requests and observe responses.

22.1 Registration Test

Test registration using valid user details. The expected result is successful creation of the user record.

22.2 Login Test

Send valid credentials to the login endpoint. The expected result is successful authentication.

22.3 Create Blog Test

Send an authenticated request to create a blog. The expected result is a new blog document in MongoDB.

22.4 Fetch Blog Test

Send a GET request to retrieve blog information. The expected result is JSON data.

22.5 AI Generation Test

Send a suitable topic to the AI endpoint. The expected result is generated content.

22.6 Summary Test

Send blog content to the summary endpoint. The expected result is a shorter summary.

22.7 Testing Checklist

- Registration
- Login and authentication
- Protected routes
- Blog creation
- Blog retrieval
- Comments and moderation
- AI blog generation
- AI summarization
- Invalid input handling

23. LIMITATIONS AND FUTURE SCOPE

23.1 Limitations

The supplied documentation mainly describes the backend API and API testing. It does not provide a complete frontend interface description, so this documentation focuses on backend services.

23.2 Future Scope

The system can be extended with a dedicated frontend, more detailed analytics, additional content-management functions and further AI workflow improvements.

23.3 Scalability

The separation of routes, controllers, models and services makes the application easier to maintain when additional features are introduced.

24. PROJECT SUMMARY

ContentGen is a Node.js and Express.js REST API project for blog content management. MongoDB is used as the database and Mongoose is used for database interaction. The system supports users, blogs, comments, categories, tags and role-based permissions.

Google Gemini AI is integrated for blog generation and summarization. JWT authentication and role-based authorization protect restricted operations. Password hashing, validation, sanitization, rate limiting and structured logging are described as security mechanisms.

The project follows a layered architecture and MVC-style organization. The model layer handles data structure, the controller handles processing and the API routing layer connects requests to controllers.

FINAL CONCLUSION

The ContentGen project demonstrates how REST APIs, Node.js, Express.js, MongoDB, Mongoose, authentication and Generative AI can be combined to build a practical backend application. It provides a centralized approach to blog management and demonstrates the use of AI in content creation.

ACKNOWLEDGEMENT

I would like to express my sincere thanks to my faculty members and project guidance team for giving me the opportunity to work on this project. Their guidance helped me understand backend development, database management, API testing and AI integration.

I also thank my classmates and everyone who supported me during the development and documentation of this project.

DECLARATION

I hereby declare that this project documentation has been prepared as part of my academic project work. It explains the ContentGen backend project, its architecture, requirements, database design, API operations, security features and AI-related functions.